Sentiment Analysis – A Comparative Study

Class: Artificial Intelligence (CSC-447-UT1, CSC-547-UT1)

Department of Computer Science

University of South Dakota

Group members:

Neerajdattu Dudam (Grad)

Teja Mallireddy (Grad)

Supervisor: Dr. Lina Chato

Spring 2025

1. Abstract

This project presents a comprehensive study of sentiment analysis techniques aimed at classifying text into positive, negative, and neutral sentiments. We explore a wide range of methodologies, including lexicon-based approaches, traditional machine learning algorithms, and advanced deep learning models. Lexicon-based methods such as VADER, AFINN, SentiWordNet, TextBlob, and WordStat are implemented to assess sentiment using predefined dictionaries. Classical machine learning techniques—including Naive Bayes, Logistic Regression, Support Vector Machines, Decision Trees, Random Forests, k-Nearest Neighbors, and Gradient Boosting Machines—are evaluated for their effectiveness in sentiment classification. In the deep learning domain, we experiment with models like Recurrent Neural Networks (RNNs) with LSTM and GRU units, Convolutional Neural Networks (CNNs), Bidirectional LSTMs, attention mechanisms, and transformer-based architectures such as BERT and GPT. Additionally, we investigate hybrid models combining CNNs and RNNs, ensemble learning strategies, and various feature engineering techniques, including TF-IDF, Word2Vec, GloVe, and Universal Sentence Encoder embeddings. The study also considers unsupervised and semi-supervised learning approaches. The primary objectives are to gain a robust understanding of Natural Language Processing (NLP), examine the practical implications of sentiment analysis, and develop proficiency in implementing and evaluating diverse sentiment analysis models. Through this project, we aim to contribute an in-depth comparative analysis that enhances both theoretical understanding and practical application in the field of sentiment analysis.

Index Terms — Sentiment Analysis, Natural Language Processing, Lexicon-Based Methods, Machine Learning, Deep Learning, Transformer Models, BERT, CNN, RNN, LSTM, GRU, Attention Mechanism, Ensemble Learning, Feature Engineering.

2. Introduction

In today's digital era, massive volumes of textual data are generated every second across various platforms—social media, review sites, emails, blogs, and academic or organizational feedback forms. Analyzing this data manually is not only time-consuming but also inefficient when dealing with large datasets. This is where Sentiment Analysis, a subfield of Natural Language Processing (NLP), becomes crucial. It aims to automatically determine the emotional tone conveyed in a piece of text, categorizing it as positive, negative, or neutral.

To illustrate a practical scenario, consider a university course where the instructor collects feedback from students at the end of the semester via an online form. Each student types in a free-text response expressing their experience with the course. While going through each individual response might be overwhelming for the instructor, a sentiment analysis tool can quickly summarize the feedback by analyzing the sentiment distribution—e.g., 65% positive, 25% neutral, and 10% negative. This gives the lecturer an immediate, high-level understanding of how students felt about the course overall. Later, they can prioritize reading the neutral and negative reviews to reflect on areas needing improvement, while skipping most of the positive feedback unless time permits. This not only saves time but also helps in targeted self-improvement and decision-making.

Sentiment analysis has widespread applications beyond education. In business, companies use it to analyze customer reviews and brand perception. In politics, it can gauge public opinion on candidates or policies. In healthcare, patient feedback can be mined for quality assessment. In social media, it helps detect trends, crises, or public mood on a specific topic.

Over the years, various techniques have been employed to perform sentiment analysis, starting from lexicon-based methods—which rely on predefined dictionaries of words and their associated sentiment scores—to traditional machine learning approaches that learn from labeled data, and finally to deep learning and transformer-based models that automatically capture contextual nuances within text. While lexicon-based methods such as VADER and SentiWordNet are interpretable and easy to use, they often fail to capture complex sentiment expressions, sarcasm, or domain-specific language. On the other hand, machine learning and deep learning techniques have shown greater flexibility and accuracy by learning patterns from data, especially when supported with effective feature engineering or embeddings.

This project undertakes a comprehensive and comparative study of these different approaches—lexicon-based, machine learning, and deep learning—by implementing a wide range of techniques such as Naive Bayes, Support Vector Machines, LSTM networks, Convolutional Neural Networks, and transformer models like BERT and GPT. We also explore hybrid and ensemble methods, along with various feature extraction techniques like TF-IDF, Word2Vec, GloVe, and sentence embeddings, to improve classification performance[1-11].

By systematically analyzing and evaluating these models, the project aims to highlight how different techniques perform under various conditions and provide insights into which methods are best suited for different types of sentiment analysis tasks.

3. Literature Review

Early approaches to sentiment analysis often leveraged lexicons—predefined dictionaries that assign sentiment scores to individual words. Hutto and Gilbert's work on VADER (Valence Aware Dictionary and sEntiment Reasoner) [1] is a seminal contribution in this category. VADER was designed specifically for social media text, capitalizing on a rule-based system that incorporates heuristics like punctuation, capitalization, and degree modifiers. The model's parsimonious design offers both interpretability and efficiency, making it especially appealing for real-time applications. Complementing this, Bing Liu's curated resources on sentiment analysis [2] serve as a comprehensive guide, outlining the strengths and limitations of lexicon-based approaches. Although these methods are robust in scenarios with well-defined vocabularies, their reliance on static dictionaries often limits their ability to adapt to context, subtle nuances, and emerging slang.

The advent of machine learning marked a significant turning point in sentiment analysis. Pang, Lee, and Vaithyanathan [3] introduced one of the earliest and most influential machine learning-based studies for sentiment classification. By applying techniques such as Naive Bayes and Support Vector Machines (SVMs), they demonstrated that statistical learning could achieve competitive performance compared to rule-based approaches. Joachims' work [4] further substantiates the utility of SVMs for text categorization, emphasizing how the algorithm's capacity to handle high-dimensional data makes it well-

suited for sentiment tasks. Additionally, Vink and de Haan [5] offered a comparative analysis of various machine learning techniques. Although their focus was on target detection within artificial intelligence applications, many of the lessons drawn—such as the impact of feature selection and algorithmic trade-offs—remain applicable to sentiment analysis.

A major development that underpinned later advances in sentiment analysis was the introduction of distributed word representations. Mikolov et al. [6] proposed models that capture semantic and syntactic relationships between words through vector embeddings (commonly known as Word2Vec). These embeddings facilitate the encoding of subtle contextual cues that traditional bag-of-words models typically miss, thereby laying the foundation for incorporating richer textual features into sentiment analysis pipelines.

Building on the success of word embeddings, researchers shifted focus to deep learning architectures that could model complex language patterns automatically. Liu, Qiu, and Huang [7] utilized recurrent neural networks (RNNs) in a multi-task learning framework to improve text classification performance, showing that neural networks can effectively capture sequential dependencies in text. Alongside RNNs, convolutional neural networks (CNNs) have also been applied to sentence-level classification. For example, Rakhlin's implementation on GitHub [8] of CNNs for sentence classification illustrates how convolutional filters can extract local, n-gram features from text, often achieving impressive results with relatively simple network architectures.

The recent breakthrough in natural language processing has been driven by transformer architectures. Devlin et al.'s introduction of BERT (Bidirectional Encoder Representations from Transformers) [9] marked a paradigm shift by enabling deep bidirectional context learning—a feature critical for nuanced sentiment classification. BERT's ability to consider the entire sentence context significantly improves the detection of sentiment nuances that earlier models might miss. In parallel, the work by Brown et al. [10] on large-scale, transformer-based language models (commonly referred to as GPT models) has pushed the envelope even further by demonstrating robust few-shot learning abilities. These models not only excel in sentiment analysis tasks but also in generating coherent, context-aware text, thereby expanding the range of applications within and beyond sentiment categorization.

While the majority of advances have relied on large volumes of labeled data, the challenge of data scarcity has also been addressed by semi-supervised and hybrid methods. Chou, Chang, and Wu [11] explore semi-supervised sequence labeling, originally in the context of Chinese person name extraction. Their tri-training framework illustrates how unlabeled data can be effectively integrated to enhance learning performance. Although applied to a different domain, the methodology has implications for sentiment analysis, where unlabeled textual data is abundant and can be leveraged to refine model training, particularly in low-resource settings.

From lexicon-based systems like VADER to deep transformer models such as BERT and GPT, the evolution of sentiment analysis techniques reflects a continuous effort to balance interpretability, scalability, and accuracy. Early rule-based approaches provide intuitive insights but fall short in handling

context, while traditional machine learning methods offer improvements through statistical learning. The integration of word embeddings and the transition to deep learning architectures have enabled the capture of more complex linguistic features, and transformer-based models have heralded a new era in capturing bidirectional context. Furthermore, the incorporation of semi-supervised strategies highlights an emerging trend towards models that can learn from both labeled and unlabeled data, potentially overcoming one of the major bottlenecks in sentiment analysis research.

4. Data

The primary dataset used for this project is dynaset-v1.1, which offers a rich and diverse text corpus tailored for sentiment analysis tasks. This dataset is designed to facilitate the training and evaluation of various sentiment classification models by providing well-labeled text samples. Dynaset-v1.1 consists of two distinct sub-datasets that complement each other in terms of natural language variety and classification difficulty:

- **Yelp Dataset:**

  The Yelp component of dynaset-v1.1 is composed of naturally occurring sentences collected from user reviews. This dataset contains real-world examples of everyday language, capturing the informal and varied expressions commonly found in review texts. The natural variability in sentence structure and vocabulary makes the Yelp dataset an excellent resource for understanding sentiment as expressed in organic and user-generated content.

- **Dynabench Dataset:**

  In contrast, the Dynabench subset is curated through crowdsourcing, focusing on collecting sentences that are inherently more challenging to classify. This dataset features text samples that are carefully selected or crafted to present subtle sentiment cues or ambiguous expressions. As a result, these sentences often require more sophisticated analysis, making this subset particularly valuable for evaluating the robustness of sentiment analysis models under less straightforward conditions.

Both sub-datasets are segmented into three partitions: training, validation, and test sets. This division allows for systematic model development and evaluation:

- The training set is used to learn the model parameters.
- The validation set provides a means to tune hyperparameters and monitor model performance during the development phase.
- The test set offers a final evaluation to assess the model's generalization capability on unseen data.

The overall distribution of the data—across the Yelp and Dynabench subsets as well as the individual partitions—is illustrated in Figure 1. This figure summarizes the quantity and diversity of samples in each subset, highlighting the balance between naturally occurring texts and carefully curated examples. By leveraging this structured dataset, the project ensures a comprehensive evaluation of sentiment analysis techniques across varying degrees of textual complexity and sentiment ambiguity.

Fig 1: Data Distribution

5. Methods

Data:

The original datasets were provided in .jsonl (JSON Lines) format, which is not directly compatible with common data manipulation tools. Therefore, the first step involved converting these files into .csv format. During this conversion, only the relevant features were retained—specifically, the sentence field containing the text and the gold_label field representing the sentiment label associated with each sentence. These labels could take on values such as positive, negative, neutral, or mixed. Following this, a cleaning process was performed where all rows with missing labels were removed. This step revealed that the dynabench training set contained 2,136 rows with missing labels, while the yelp training set had 10,071 such rows. After removing these, the datasets were checked for additional missing entries and duplicates, of which none were found. The state of the dataset after these preprocessing steps is shown in Figure 2.

file_name	total_rows	positive_rows	negative_rows	mixed_rows	neutral_rows
dynasent-dynabench-dev.csv	720	240	240	0	240
dynasent-dynabench-test.csv	720	240	240	0	240
dynasent-dynabench-train.csv	16399	6038	4579	3334	2448
dynasent-yelp-dev.csv	3600	1200	1200	0	1200
dynasent-yelp-test.csv	3600	1200	1200	0	1200
dynasent-yelp-train.csv	84388	21391	14021	3900	45076

Fig 2: Class-wise dataset distribution

Upon closer inspection, it was observed that the 'mixed' sentiment label appeared exclusively in the training sets and was absent in the test and validation sets of both datasets. Since the inconsistency in label presence could pose issues during model evaluation, all rows labeled as 'mixed' were removed from the training sets. To further enhance the study, a new dataset was constructed by combining the Yelp and Dynabench datasets, thereby leveraging both naturally occurring text (from Yelp) and carefully crowdsourced, often harder-to-classify sentences (from Dynabench). The class distribution of this combined dataset is illustrated in Figure 3. Although the test and validation sets were reasonably balanced across sentiment classes, the training sets displayed significant imbalance. To address this, the Yelp training set was downsampled to 14,021 samples per sentiment class, with this number corresponding to the count of the minority class (negative). Similarly, the Dynabench training set was downsampled to 2,448 samples per class, as the neutral class in this set had the lowest representation.

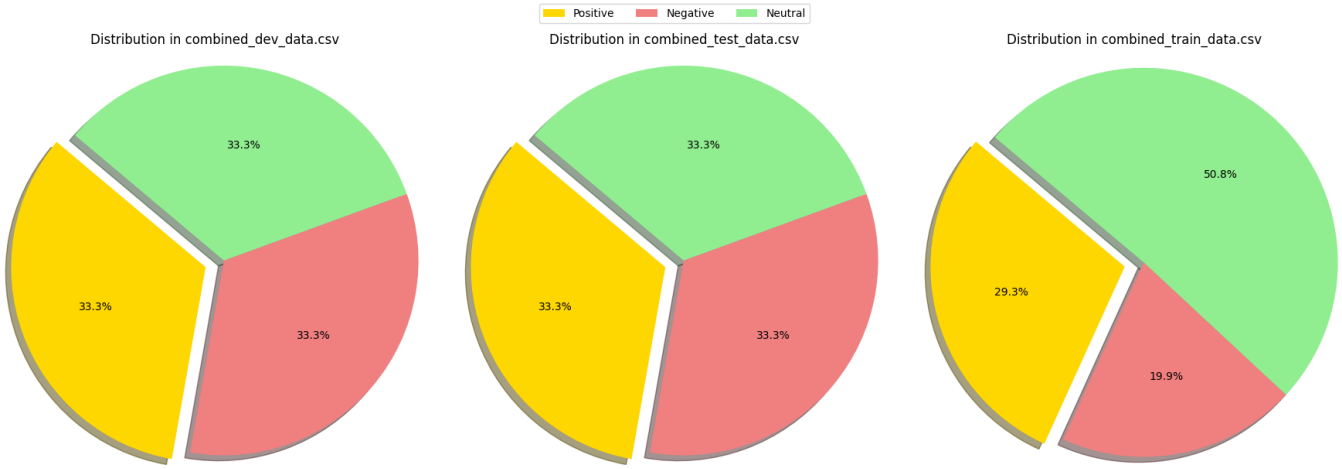


Fig 3: Data Distribution of combined dataset

After preprocessing, three finalized datasets were established. The Yelp dataset consisted of training, test, and validation sets. In the training set, each sentiment class (positive, negative, and neutral) contained 14,021 samples. Similarly, in the test and validation sets, each class had 1,200 samples. The Dynabench dataset also followed the same structure, with each sentiment class having 2,448 samples in the training set and 240 samples each in the test and validation sets. Lastly, the combined dataset, formed by merging both individual datasets, contained 16,469 samples per class in the training set and 480 samples per class in both the test and validation sets. The primary motivation behind this final dataset was to evaluate whether a model trained on a mixture of both naturally occurring and adversarially constructed samples could exhibit improved generalization and robustness in real-world sentiment analysis tasks.

Methodology:

To comprehensively evaluate sentiment classification, we employed a wide range of modeling strategies, categorized into four broad families: lexicon-based methods, traditional machine learning models, deep learning architectures, and ensemble approaches. Lexicon-based methods involve rule-based techniques that use pre-defined sentiment dictionaries to assign polarity scores to text. In this study, we utilized popular lexicons such as VADER, AFINN, SentiWordNet, and TextBlob, each offering unique scoring strategies for capturing sentiment nuances. Moving beyond rule-based systems, we implemented classical machine learning algorithms that learn from labeled data. Models such as Naive Bayes, Logistic Regression, Support Vector Machines (SVM), Decision Trees, and Random Forests were trained using features extracted from text using TF-IDF or Bag-of-Words representations, depending on the model requirements.

To capture deeper semantic and contextual features, we explored various deep learning models, starting with Recurrent Neural Networks (RNNs) including Long Short-Term Memory (LSTM) and Gated Recurrent Units (GRU), which are particularly suited for sequential data like text. We also experimented with Convolutional Neural Networks (CNNs) applied over n-gram representations to detect local patterns in sentences. In addition, transformer-based models such as BERT and GPT were incorporated to leverage contextual embeddings and attention mechanisms for improved performance. A Bi-directional LSTM (BiLSTM) was also implemented to capture dependencies from both past and future tokens in a sentence, further enhancing the model's understanding of context. Lastly, ensemble methods were adopted to combine the strengths of individual models through techniques like voting and stacking, thereby aiming to achieve more robust and generalized predictions. Each model was trained and tested on the datasets prepared during preprocessing, ensuring consistent evaluation across all techniques.

Training and Evaluation:

All models were trained using the respective training sets for Yelp, Dynabench, and the combined dataset. During training, we monitored performance using metrics such as accuracy and confusion matrices. The testing phase involved applying the trained models to the test sets and evaluating their predictions using confusion matrices and accuracy scores.

Finally, a comparative analysis was conducted across all models to assess their performance on both individual and combined datasets. This allowed us to evaluate whether models trained on both natural and adversarial sentences performed better than those trained on a single source.

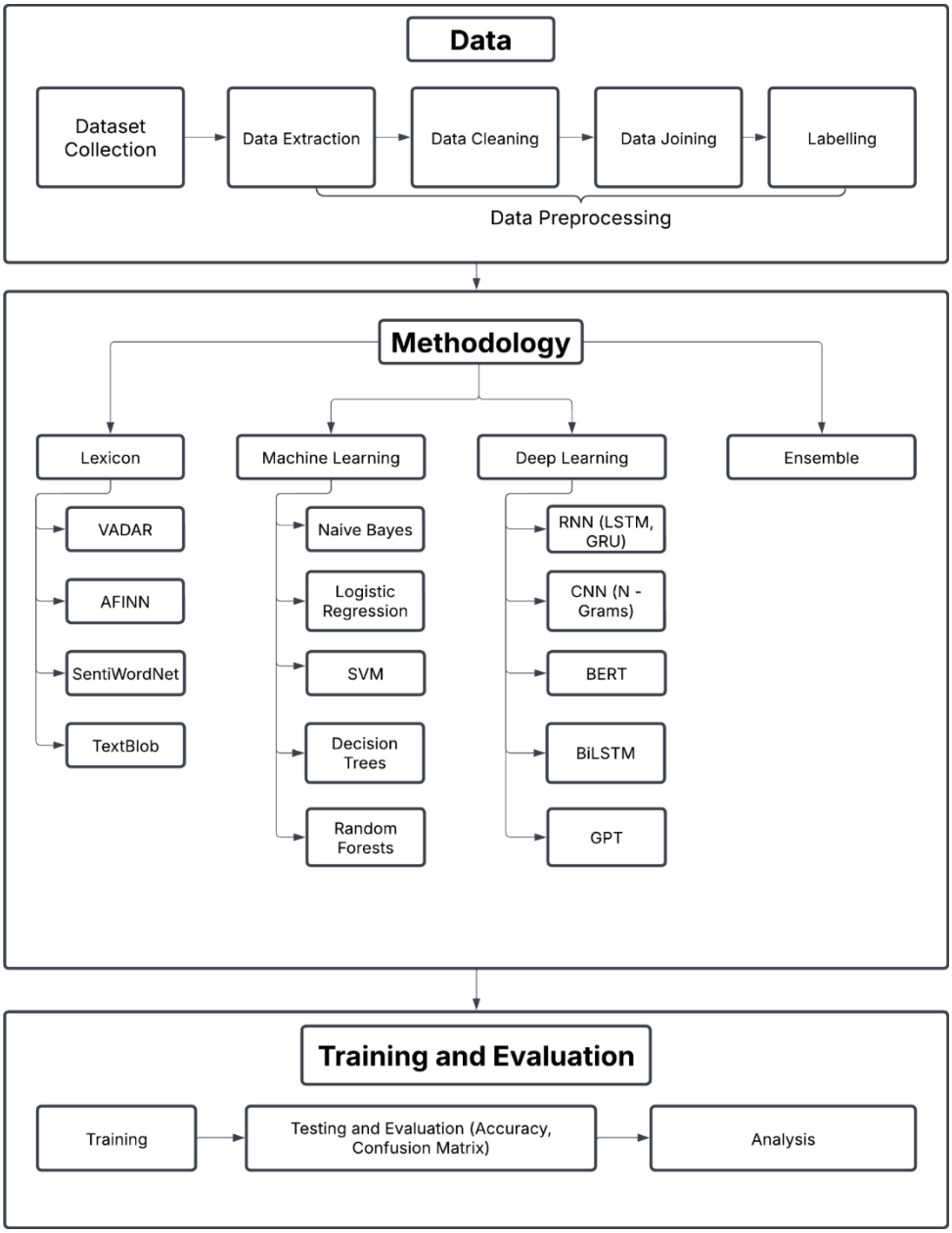


Fig 2: Workflow and methods

6. Experiments

In this section, we present the experimental setup and methodologies used for sentiment classification. The objective was to compare lexicon-based, machine learning-based, and deep learning-based sentiment analysis techniques using a consistent and well-preprocessed dataset. All experiments were conducted after applying a uniform text preprocessing pipeline which included lowercasing, punctuation removal, stop word removal, and most importantly, lemmatization. Lemmatization reduced words to their

dictionary forms, thereby improving semantic consistency across models and reducing noise in feature extraction.

We conducted twelve primary experiments: the first four used lexicon-based sentiment analysis techniques, and the remaining eight utilized traditional machine learning models. For all traditional machine learning models, we relied on scikit-learn's implementation, except for one manually implemented Naive Bayes classifier from scratch. In the logistic regression model, we further evaluated three different feature extraction strategies. All models were trained and evaluated on the same data splits for fairness.

Lexicon Based Approach

Experiment 1: (Using VADER)

The first experiment employed the VADER (Valence Aware Dictionary and sEntiment Reasoner), which is a lexicon and rule-based sentiment analysis tool, particularly optimized for microtext like tweets. Each word in the VADER lexicon is assigned a valence score between -4 (most negative) and +4 (most positive). The sentiment score of a sentence is computed by summing these valence scores and adjusting them based on grammatical heuristics such as punctuation (e.g., exclamation marks), degree modifiers (e.g., "very"), negations (e.g., "not good"), and even the presence of emojis or all-uppercase words. The final result is normalized to obtain a compound score ranging from -1 to +1, which is then mapped to positive, neutral, or negative sentiment classes.

$$Compound\ Score = \frac{\sum_i s_i}{\sqrt{\sum_i s_i^2 + \alpha}}$$

Here, s_i is the valence score of the i-th word and α is a normalization factor (default is 15).

Experiment 2: (Using AFINN)

The second experiment used the AFINN lexicon, which assigns integer scores to words ranging from -5 to +5. The overall sentiment of a sentence is computed as the sum of the scores of all matching words in the text. Although it is a simple approach without syntactic awareness, it is intuitive and computationally efficient. However, it does not account for contextual modifiers, negations, or word order.

$$Sentiment\ Score = \sum_{i=1}^{n} score(w_i)$$

where, w_i is aword in the input text.

Experiment 3: (Using SentiWordNet)

The third experiment utilized SentiWordNet, an extension of WordNet in which each synset (group of synonyms) is assigned three sentiment scores: positivity, negativity, and objectivity. Since a single word can belong to multiple synsets, we calculated the word's sentiment score as the average of the differences between positive and negative scores across all its synsets.

$$Sentiment\ Score\ (w) = \frac{1}{N}\sum_{i=1}^{N}(Pos_i(w) - Neg_i(w))$$

where N is the number of synsets the word w appears in. This method allows for more nuanced scoring but depends heavily on accurate word sense disambiguation.

Experiment 4: (Using TextBlob)

The fourth experiment applied TextBlob, a lexicon-based model that combines rule-based parsing and a sentiment lexicon derived from Pattern and NLTK. It outputs two values: polarity (ranging from -1 to +1) and subjectivity (ranging from 0 to 1). The sentiment of each sentence is determined based on the average polarity score of relevant words, after POS tagging filters out irrelevant parts of speech.

Machine Learning Based Approach

Experiment 5: (Naive Bayes from Scratch)

In this implementation, we manually computed the class priors, word likelihoods, and prediction probabilities. This approach provided a deep understanding of how the model works internally, including the impact of rare words and class imbalance.

$$P(c,d) = \frac{P(c)\prod_{i=1}^{n}P(w_i|c)}{P(d)}$$

Since P(d) is constant across all classes, we used the log-likelihood:

$$\hat{y} = argmax_c\ (\log P(c) + \sum_{i=1}^{n} log\ P(w_i|c))$$

where P(c): prior probability of class c, $P(w_i \mid c)$: likelihood of word w_i given class c, n: number of words in the document

To prevent zero probabilities, Laplace smoothing was applied:

$$P(w_i|c) = \frac{N_{w_{i,c}} + 1}{\sum_j N_{w_{i,c}} + |V|}$$

where $N_{w_{i,c}}$ is the count of w_i in class c, and |V| is the vocabulary size. This manual implementation allowed us to clearly visualize the effect of class priors and word frequencies on the final prediction.

Experiment 6: (Multinomial Naive Bayes with CountVectorizer)

In this experiment, we implemented the Multinomial Naive Bayes classifier using CountVectorizer as the feature extractor. CountVectorizer tokenizes the text and creates a feature vector representing the frequency of each word in the document, without accounting for global document frequency. Given the multi-class nature of our dataset (positive, neutral, and negative), we used the Multinomial variant of Naive Bayes.

Experiment 7: (Logistic Regression with Frequency-Based Features)

The seventh experiment involved a Logistic Regression classifier using frequency-based feature extraction. Each word's frequency in a document was used as the feature. The model was trained using the Adam optimizer, with a learning rate of 0.001 for 100 epochs. The logistic regression model estimates the probability of a class label using the sigmoid function:

$$P(y = 1 \mid x) = \frac{1}{1 + e^{-(w^T x + b)}}$$

The model parameters w, b are learned by minimizing the binary cross-entropy loss. However, since we were working with three classes, we employed a softmax-based multi-class logistic regression model.

Experiment 8: (Logistic Regression with CountVectorizer Features)

The eighth experiment used Logistic Regression with CountVectorizer as the feature extraction technique. The setup remained the same as the previous experiment, with the Adam optimizer, 100 epochs, and a learning rate of 0.001. CountVectorizer was simpler but often effective for short texts.

Experiment 9: (Logistic Regression with TF-IDF Features)

The ninth experiment used TF-IDF (Term Frequency-Inverse Document Frequency) features in Logistic Regression. TF-IDF balances the term frequency of a word in a document with its rarity across the corpus, thereby reducing the impact of commonly occurring words like "the" and "is". The training parameters (optimizer, epochs, learning rate) were kept constant for consistency and comparative evaluation.

Experiment 10: (Support Vector Machine (SVM) with TF-IDF Features)

In the tenth experiment, we implemented a Support Vector Machine (SVM) classifier using TF-IDF features and a linear kernel. SVM finds the optimal separating hyperplane that maximizes the margin between the classes. The decision function is defined as:

$$f(x) = w^T x + b$$

The optimization objective minimizes the norm of the weight vector while ensuring that each data point lies on the correct side of the margin. Given the sparse and high-dimensional nature of TF-IDF vectors, SVM is particularly effective and often yields strong generalization performance.

Experiment 11: (Decision Tree Classifier with TF-IDF Features)

The eleventh experiment utilized a Decision Tree Classifier, again with TF-IDF features. Decision Trees recursively split the dataset based on the feature that provides the highest information gain or minimizes Gini impurity. The Gini impurity at a node is defined as:

$$G(t) = 1 - \sum_{i=1}^{C} p_i^2$$

where p_i is the proportion of samples belonging to class i at node t. Decision Trees are inherently interpretable and work well on smaller datasets but are prone to overfitting.

Experiment 12: (Random Forest Classifier with TF-IDF Features)

Finally, the twelfth experiment implemented a Random Forest classifier with TF-IDF features. Random Forests are ensemble methods that aggregate the predictions of multiple Decision Trees trained on bootstrapped subsets of the data, with random subsets of features considered at each split. This reduces variance and improves robustness. The final class prediction is made by majority voting across the individual trees.

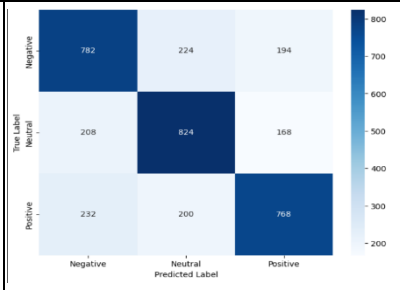
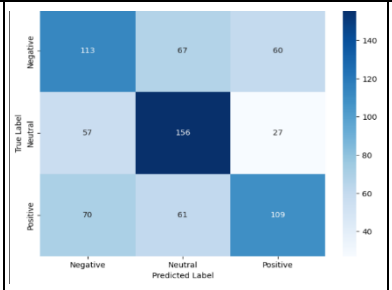
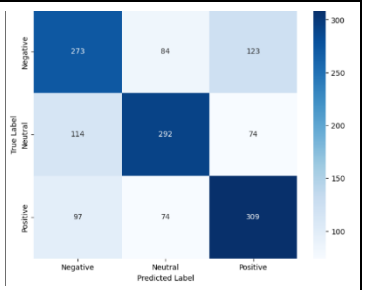
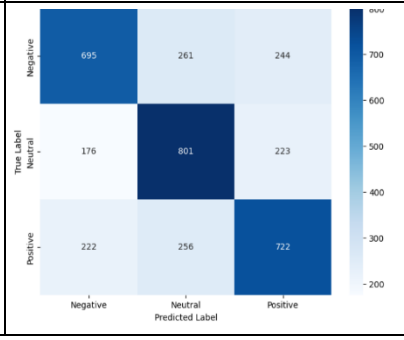
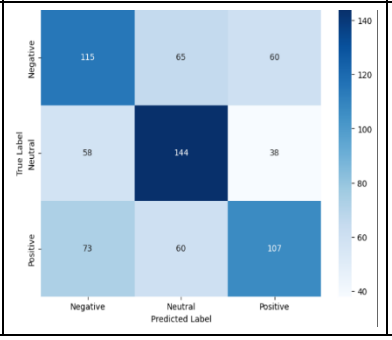
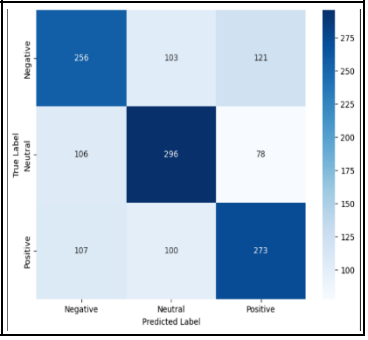
7. Results

Test Accuracy (%):

Exp No.	Experiment Name	Yelp Dataset	Dynabench Dataset	Combined Dataset
1	Using VADER	53	57	53
2	Using AFINN	52	57	53
3	Using SentiWordNet	43	44	43
4	Using TextBlob	50	54	51
5	Naive Bayes from Scratch	64	52	57
6	Multinomial Naive Bayes	65	53	57
7	Logistic Regression 1	54	49	45
8	Logistic Regression 2	60	50	56
9	Logistic Regression 3	62	51	57
10	SVM	66	53	61
11	Decision Tree	55	49	52
12	Random Forest	62	54	59

Confusion Matrices:

There were many confusion matrices that we built, we as part of the report we are reporting the best matrices that we got from SVM and logistic regression with TF-IDF features.

Exp Name	Yelp Dataset	Dynabench Dataset	Combined Dataset
SVM			
Logistic Regression with TF-IDf features			

8. Conclusions and Discussion

The comparative results from twelve distinct experiments conducted on three datasets—Yelp, Dynabench, and a Combined Dataset—highlight key insights into the strengths and limitations of various sentiment analysis approaches. The experiments span from lexicon-based methods to traditional machine learning models, all evaluated on a consistent lemmatized preprocessing pipeline.

The first four experiments utilized lexicon-based models: VADER, AFINN, SentiWordNet, and TextBlob. Among these, VADER and AFINN showed relatively better performance, with VADER achieving the highest accuracy of 57% on the Dynabench dataset. Their simplicity and domain independence make them useful for quick sentiment analysis tasks, especially when labeled training data is scarce. However, these models rely heavily on word-level polarity scores and cannot account for contextual or syntactic nuances. SentiWordNet performed the worst overall, with accuracies as low as 43% on both Yelp and Combined datasets. This can be attributed to its dependence on correct word-sense disambiguation and its lack of handling for modifiers and negations. TextBlob, while performing better than SentiWordNet, still did not surpass the 54% mark on any dataset, indicating the limited generalizability of these rule-based systems.

In contrast, machine learning-based methods offered significant improvements. Both Multinomial Naive Bayes (Experiment 6) and the Naive Bayes from scratch implementation (Experiment 5) outperformed all lexicon-based models. Notably, the Multinomial Naive Bayes model reached 65% accuracy on the Yelp dataset and 57% on the Combined dataset. This demonstrates the effectiveness of statistical learning and feature-based modeling for sentiment classification, particularly when textual patterns are captured through tokenization and frequency-based features. Interestingly, the scratch implementation performed

slightly worse, especially on the Dynabench dataset (52%), likely due to the lack of sophisticated preprocessing and regularization features present in scikit-learn's optimized implementation.

Experiments 7 through 9 investigated the impact of different feature extraction methods within Logistic Regression models. The third variation (using TF-IDF) performed best, achieving 62% on Yelp and 57% on the Combined dataset, suggesting that weighting terms by their importance across documents improves discriminative power. On the other hand, the frequency-based logistic model (Experiment 7) performed poorly, especially on the Combined dataset (45%), underscoring the limitations of raw frequency features that can overemphasize common terms. The count vectorizer-based model (Experiment 8) provided a reasonable balance with 60% accuracy on Yelp. Overall, these results emphasize the significance of feature engineering in influencing model performance, especially in text classification.

The Support Vector Machine (SVM) classifier (Experiment 10) yielded the best overall performance on the Yelp (66%) and Combined (61%) datasets, reaffirming the efficacy of margin-based classifiers in high-dimensional, sparse vector spaces such as TF-IDF. SVM's robustness to overfitting and its ability to find optimal separating hyperplanes make it highly suitable for sentiment classification tasks with diverse linguistic structures.

The Decision Tree and Random Forest models (Experiments 11 and 12) showed mixed results. While Decision Trees were relatively weak performers with accuracies as low as 49%, Random Forests delivered competitive results—62% on Yelp and 59% on the Combined dataset. The ensemble nature of Random Forests helps overcome overfitting and enhances generalization compared to single-tree models. However, they still fell short of SVM in performance, suggesting that while ensemble models help, they may not always capture the linear separability found in TF-IDF-transformed spaces as effectively as SVMs.

A notable trend across all models was that performance tended to drop slightly on the Dynabench dataset compared to Yelp. This indicates potential domain or annotation differences between the datasets, with Dynabench possibly containing more nuanced or ambiguous sentiment expressions. Nonetheless, performance on the Combined dataset often mirrored or averaged the individual results, which highlights the stability and adaptability of well-regularized machine learning models across varied textual domains.

In conclusion, the experiments clearly demonstrate that machine learning models significantly outperform lexicon-based methods in sentiment classification, particularly when paired with sophisticated feature extraction techniques like TF-IDF. SVM emerged as the most effective model overall, achieving the highest accuracies on two of the three datasets. Among lexicon-based models, VADER remains a strong baseline due to its simple yet effective handling of microtext sentiment. Furthermore, the impact of feature engineering was evident within logistic regression models, where TF-IDF provided consistent gains over count or frequency-based approaches. These findings reinforce the idea that for domain-general and high-performing sentiment classification, data-driven supervised models paired with context-aware feature engineering are the most reliable choice.

9. Future work

In this study, we focused on two primary approaches for sentiment classification: lexicon-based methods and traditional machine learning models. While these techniques provided a solid baseline and helped us understand the behavior of the datasets, we did not explore deep learning methods as part of this work. As a natural extension, our future work will involve evaluating the performance of our datasets using various deep learning architectures.

Specifically, we aim to implement models such as CNN with n-gram features, LSTM, GRU, BERT, and GPT, which are known to achieve state-of-the-art results in sentiment analysis. These models are capable of capturing complex linguistic patterns, semantic context, and long-range dependencies, which traditional models may miss. Through this, we plan to assess whether deep learning methods can offer significant improvements in accuracy and generalization over the current approaches, especially when applied to a diverse and balanced dataset like the one we constructed.

By incorporating deep learning into our workflow, we hope to build a more comprehensive and powerful sentiment analysis pipeline that complements and surpasses our current results.

10. References

[1]. C. Hutto and E. Gilbert, "VADER: A parsimonious rule-based model for sentiment analysis of social media text," Proc. Int. AAAI Conf. Web Soc. Media, vol. 8, no. 1, 2014.

[2] B. Liu, "Sentiment analysis," UIC, [Online]. Available: https://www.cs.uic.edu/~liub/FBS/sentiment-analysis.html. [Accessed: Feb. 6, 2025].

[3] B. Pang, L. Lee, and S. Vaithyanathan, "Thumbs up? Sentiment classification using machine learning techniques," arXiv preprint cs/0205070, 2002.

[4] T. Joachims, "Text categorization with support vector machines: Learning with many relevant features," in Proc. Eur. Conf. Mach. Learn., Berlin, Heidelberg: Springer, 1998, pp. 137–142.

[5] J. P. Vink and G. de Haan, "Comparison of machine learning techniques for target detection," Artif. Intell. Rev., vol. 43, pp. 125–139, 2015.

[6] T. Mikolov et al., "Distributed representations of words and phrases and their compositionality," in Adv. Neural Inf. Process. Syst., vol. 26, 2013.

[7] P. Liu, X. Qiu, and X. Huang, "Recurrent neural network for text classification with multi-task learning," arXiv preprint arXiv:1605.05101, 2016.

[8] A. Rakhlin, "Convolutional neural networks for sentence classification," GitHub, vol. 6, p. 25, 2016.

[9] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, "BERT: Pre-training of deep bidirectional transformers for language understanding," in Proc. NAACL-HLT, vol. 1, no. 2, 2019, pp. 4171–4186.

[10] T. Brown et al., "Language models are few-shot learners," in Adv. Neural Inf. Process. Syst., vol. 33, 2020, pp. 1877–1901.

[11] C.-L. Chou, C.-H. Chang, and S.-Y. Wu, "Semi-supervised sequence labeling for named entity extraction based on tri-training: case study on Chinese person name extraction," in Proc. Third Workshop Semantic Web Inf. Extraction, 2014.